Homework 3 — Solutions

CS-UY 2413: Design & Analysis of Algorithms

Fall 2026 • Prof. Lisa Hellerstein

1. Karatsuba's algorithm for fast integer multiplication can also be applied to decimal numbers. The pseudocode is the same, except that we replace $2^{n/2}$ and 2^n by $10^{n/2}$ and 10^n.

Using the decimal version of Karatsuba's algorithm, step through just the initial call of the algorithm, multiplying 87654321 by 23456789, as described in class: Show all the steps, but for each recursive call, just show the inputs (numbers to multiply) and the result (their product), without going farther down the recursion tree.

Solution. Both numbers have $n = 8$ digits, so we split each into halves of $n/2 = 4$ digits:

$$x = x_1 \cdot 10^4 + x_0, \quad y = y_1 \cdot 10^4 + y_0, \qquad x_1 = 8765,\ x_0 = 4321,\ y_1 = 2345,\ y_0 = 6789.$$

Then $xy = x_1y_1 \cdot 10^8 + (x_1y_0 + x_0y_1) \cdot 10^4 + x_0y_0$, and Karatsuba gets the middle term from one extra multiplication using $x_1y_0 + x_0y_1 = (x_1 + x_0)(y_1 + y_0) - x_1y_1 - x_0y_0$.

Additions: $x_1 + x_0 = 13086$ and $y_1 + y_0 = 9134$.
Recursive calls:

$$\begin{aligned} P_1 &= x_1y_1 = 8765 \cdot 2345 = 20{,}553{,}925, \\ P_2 &= x_0y_0 = 4321 \cdot 6789 = 29{,}335{,}269, \\ P_3 &= (x_1 + x_0)(y_1 + y_0) = 13086 \cdot 9134 = 119{,}527{,}524. \end{aligned}$$

Middle term: $P_3 - P_1 - P_2 = 119{,}527{,}524 - 20{,}553{,}925 - 29{,}335{,}269 = 69{,}638{,}330$.
Combine:

$$\begin{aligned} xy &= 20{,}553{,}925 \cdot 10^8 + 69{,}638{,}330 \cdot 10^4 + 29{,}335{,}269 \\ &= 2{,}055{,}392{,}500{,}000{,}000 + 696{,}383{,}300{,}000 + 29{,}335{,}269 = 2{,}056{,}088{,}912{,}635{,}269. \end{aligned}$$

2. When we gave the running time of the priority queue operations in class, for the doubly-linked list and heap implementations, we excluded the time for mapping operations. In this problem we consider the heap implementation of the priority queue and the Increase Key operation, which locates x in the heap and then bubbles it up using swaps.

(a) Under Handle A, what is the worst case cost of updating the handles for the two elements involved in a single swap during bubble-up? Summing over the whole bubble-up, what is the overall worst case runtime of Increase-Key?

Solution. Under Handle A, updating the handles in a swap is $O(1)$, and using the handle to locate x in the heap is $O(1)$. The bubble-up performs $O(\log n)$ swaps, so

$$T(n) = O(1) + O(\log n) \cdot O(1) = O(\log n).$$

Answer: $O(1)$ per swap; $O(\log n)$ overall in the worst case.

(b) Under Handle B, what is the average overall runtime of Increase-Key (including the initial lookup of x and all handle updates during bubble-up)? What is the worst case overall runtime?

Solution. There is one hash table lookup to locate x in the heap, plus two handle updates per swap, giving $1 + 2\log n = O(\log n)$ hash operations.

Average case: a hash operation takes $O(1)$ time, so

$$O(1) \cdot O(\log n) = O(\log n).$$

Worst case: every key collides and a hash operation takes $O(n)$ time, so

$$O(n) \cdot O(\log n) = O(n \log n).$$

Answer: $O(\log n)$ on average; $O(n \log n)$ in the worst case.

(c) In class we simplified the runtime analysis of Increase-Key, using the heap implementation of the PQ, by excluding the time spent in mapping operations. Does it make sense to make this simplification if Handle A is used? If Handle B is used? Why or why not?

Solution. Handle A: Yes, the simplification is fine, since the mapping is $O(1)$ and the bound stays $O(\log n)$.

Handle B: Only for the average case. In the worst case the mapping cost $O(n \log n)$ dominates the $O(\log n)$ of heap work, so it cannot be excluded.

Answer: Yes for Handle A; for Handle B, only in the average case.

3. Suppose we insert n elements into an initially empty priority queue, implemented as a max-heap, using Max-Heap-Insert.

(a) Suppose $n = 4$ and the elements are inserted with keys $2, 7, 1, 8$, in that order. What does the heap look like after each of the four insertions? Draw the heap as a tree.

Solution.

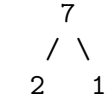
After inserting 2:



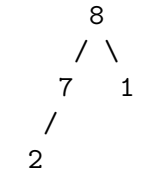
After inserting 7 (swap with 2):



After inserting 1 (no swap):



After inserting 8 (swap with 2, then with 7):



(b) For large n, will the n insertions be faster if the elements are inserted in increasing key order, or in decreasing key order? As a function of n, what is the runtime for each case? Answer using Θ notation.

Solution.

Increasing order: Each newly inserted element is larger than all previous elements, so it bubbles up to the root. The i-th insertion takes $\Theta(\log i)$ time, so we can represent that time as $c \log i$ for some constant $c > 0$. Thus, summing over the insertions $i = 2, \ldots, n$,[1]

$$T(n) = \sum_{i=2}^{n} c \log i = c \sum_{i=2}^{n} \log i.$$

We now show that this sum is $\Theta(n \log n)$.

Upper bound. Since $\log i \leq \log n$ for every $i \leq n$,

$$T(n) = c \sum_{i=2}^{n} \log i \;\leq\; c\,(n-1) \log n \;=\; O(n \log n).$$

Lower bound. At least $n/2$ of the terms in the sum (those with $i \geq n/2$) satisfy $\log i \geq \log(n/2)$, and all terms are nonnegative, so

$$T(n) \;\geq\; c \cdot \frac{n}{2} \log \frac{n}{2} = c \cdot \frac{n}{2} (\log n - 1) \;=\; \Omega(n \log n).$$

Alternatively, since $\log 1 = 0$,

$$\sum_{i=2}^{n} \log i = \log 1 + \log 2 + \cdots + \log n = \log(n!),$$

and $\log(n!) = \Theta(n \log n)$.

Therefore,

$$T(n) = \Theta(n \log n).$$

Decreasing order: Each newly inserted element is smaller than its parent, so no swaps are necessary. Each insertion takes $\Theta(1)$ time. Thus,

$$T(n) = n \cdot \Theta(1) = \Theta(n).$$

Therefore, decreasing order is faster.

[1] We start the summation at $i = 2$ because the first insertion goes into an empty heap and involves no comparisons or swaps. Starting at $i = 1$ would not change the sum anyway, since $\log 1 = 0$.

4. Suppose we wanted to add an extra operation, Max-Heap-Delete(A, x), to a Max-Heap Priority-Queue. It removes x from the heap.

(a) Give pseudocode for Max-Heap-Delete. The first step of your pseudocode should be "Map x to its position i in array A." In your pseudocode, you can call any of the Max-Heap functions defined in class and in the textbook.

Solution. We overwrite x's slot with the last element and shrink the heap by one. The moved element may now violate the heap property at position i, but only in one direction:

- if it is larger than its parent, it moves up (HEAP-INCREASE-KEY);
- otherwise it may be smaller than a child, so it moves down (MAX-HEAPIFY).

If x was itself the last element, nothing else needs to be done.

```
1:  function MAX-HEAP-DELETE(A, x)
2:      Map x to its position i in array A
3:      A[i] ← A[A.heap-size]                    ▷ overwrite x with the last element
4:      A.heap-size ← A.heap-size − 1
5:      if i ≤ A.heap-size then                  ▷ else x was the last element
6:          if i > 1 and A[i] > A[PARENT(i)] then
7:              HEAP-INCREASE-KEY(A, i, A[i])    ▷ bubble up
8:          else
9:              MAX-HEAPIFY(A, i)                ▷ bubble down
10:         end if
11:     end if
12: end function
```

Passing $A[i]$ as the new key is valid, since it is not smaller than the current key; the call simply bubbles the element up.

(b) What is the worst-case running time of your procedure, excluding the time taken for the mapping operations?

Solution. Lines 3–5 take $\Theta(1)$. Exactly one of HEAP-INCREASE-KEY or MAX-HEAPIFY is called. Each moves the element along a single path toward the root or toward a leaf, doing $\Theta(1)$ work per level of a heap of height $\lfloor \log_2 n \rfloor$. Thus

$$T(n) = \Theta(1) + \Theta(\log n) = \Theta(\log n).$$

5. Suppose you are given an array $A[1 \ldots n]$. Each entry i of the array contains a pair of integers $[x_i, y_i]$, representing the set of integers in the interval between x_i and y_i, inclusive. The *overlap* between $[x_i, y_i]$ and $[x_j, y_j]$ is defined to be the intersection of their corresponding sets. The array is sorted by the y_i values, so $y_1 < y_2 < \cdots < y_n$. Assume that the y_i values of all the pairs are distinct. The problem is to determine a pair of entries, $[x_i, y_i]$ and $[x_j, y_j]$ where $i \neq j$, such that their overlap is as large as possible.

(a) Explain how to compute the size of the overlap of a pair of entries, $[x_i, y_i]$ and $[x_j, y_j]$, in constant time.

Solution. Entry i is the set $\{z : x_i \le z \le y_i\}$, so an integer z is in the overlap exactly when it lies in both sets, that is

$$x_i \le z \le y_i \quad \text{and} \quad x_j \le z \le y_j.$$

Both pairs of inequalities hold at once exactly when

$$\max(x_i, x_j) \;\le\; z \;\le\; \min(y_i, y_j),$$

so the overlap is itself an interval of integers, running from $\max(x_i, x_j)$ to $\min(y_i, y_j)$.

An interval of integers from L to R contains $R - L + 1$ integers when $R \ge L$, and is empty otherwise. Therefore

$$\text{overlap}(i, j) \;=\; \max\big(0,\; \min(y_i, y_j) - \max(x_i, x_j) + 1\big).$$

Computing this reads four array values and performs two comparisons for the min and max, one subtraction, one addition, and one final comparison against 0. That is a constant number of operations, so it takes $\Theta(1)$ time.

As a check, $[4, 8]$ and $[5, 10]$ give $\min(y_i, y_j) = 8$ and $\max(x_i, x_j) = 5$, so $8 - 5 + 1 = 4$, which is $|\{5, 6, 7, 8\}|$.

Answer: $\text{overlap}(i, j) = \max\big(0, \min(y_i, y_j) - \max(x_i, x_j) + 1\big)$, computable in $\Theta(1)$ time.

(b) A simple algorithm takes every pair of entries $[x_i, y_i]$ and $[x_j, y_j]$ in the array, and for each pair computes the size of their overlap in constant time. Then it outputs the pair with the highest overlap. What is the worst-case running time of this algorithm, in Theta notation?

Solution. The algorithm examines every unordered pair of distinct entries, and the number of such pairs is

$$\binom{n}{2} = \frac{n(n-1)}{2} = \Theta(n^2).$$

By part (a) each pair costs $\Theta(1)$ to evaluate, and comparing the result against the running maximum is another $\Theta(1)$, so the cost per pair is $\Theta(1)$. Hence

$$T(n) = \Theta(n^2) \cdot \Theta(1) = \Theta(n^2).$$

The algorithm examines all pairs on every input, so this is both the worst case and the best case.

Answer: $\boxed{\Theta(n^2)}$.

(c) Suppose we try to solve this problem by dividing the array in half, recursively finding the pair with the largest overlap on each side, and returning whichever of the two pairs had the larger overlap. Why doesn't this work, even if we add a suitable base case?

Solution. Every pair the algorithm evaluates has both of its entries inside the same half, because the only overlaps computed are the ones returned by the two recursive calls. A pair with one interval in the left half and one interval in the right half is never considered at any level of the recursion. If the maximum overlap is attained by such a crossing pair, the algorithm cannot find it. This is the same failure as the divide and conquer for maximum subarray sum when the crossing case is omitted.

For a concrete counterexample take $n = 4$ with

$$A = \big[\,[0, 10],\ [50, 60],\ [5, 100],\ [200, 300]\,\big],$$

whose y values $10 < 60 < 100 < 300$ are increasing and distinct, as required. The left half gives

$$\text{overlap}\big([0, 10], [50, 60]\big) = \max(0,\ 10 - 50 + 1) = 0,$$

and the right half gives

$$\text{overlap}\big([5, 100], [200, 300]\big) = \max(0,\ 100 - 200 + 1) = 0,$$

so the algorithm reports a maximum overlap of 0. But the crossing pair $[50, 60]$ and $[5, 100]$ has

$$\text{overlap} = \max\big(0,\ \min(60, 100) - \max(50, 5) + 1\big) = 60 - 50 + 1 = 11,$$

so the true maximum is 11 and the algorithm is wrong.

Answer: It ignores all crossing pairs, and the best pair may be a crossing pair.

(d) Design a divide and conquer algorithm for this problem that achieves a worst-case running time of $O(n \log n)$, using the fact that the input array is sorted by y_i values and that the y_i values are all distinct. Describe the algorithm and explain why it works.

Solution.

Two facts about crossing pairs. Split the subarray $A[lo \ldots hi]$ at $m = \lfloor (lo + hi)/2 \rfloor$, so the left half is $L = A[lo \ldots m]$ and the right half is $R = A[m + 1 \ldots hi]$. A crossing pair has $i \in L$ and $j \in R$, so $i < j$ always.

First, since the array is sorted by y and the y values are distinct, $i < j$ gives $y_i < y_j$, so $\min(y_i, y_j) = y_i$ and the formula from part (a) becomes

$$\text{overlap}(i, j) \ = \ \max\big(0,\ y_i - \max(x_i, x_j) + 1\big) \qquad \text{for every crossing pair } (i, j). \qquad (*)$$

Second, fix $i \in L$ and ask which $j \in R$ maximizes $(*)$. The only part of $(*)$ that depends on j is $\max(x_i, x_j)$, and it appears with a minus sign, so maximizing the overlap means minimizing $\max(x_i, x_j)$. Since $\max(x_i, \cdot)$ is nondecreasing in its argument,

$$\min_{j \in R} \max(x_i, x_j) \ = \ \max\Big(x_i,\ \min_{j \in R} x_j\Big),$$

so the minimum is attained at the entry of R with the smallest x value. Call its index j^*. The crucial point is that j^* does not depend on i: the *same* j^* is a best partner for every $i \in L$ simultaneously. So the best crossing pair is found by fixing j^* once and scanning the left half.

Algorithm. The top-level call is Best-Overlap$(A, 1, n)$, which returns a pair together with the size of its overlap.

```
1: function Best-Overlap(A, lo, hi)
2:     if hi ≤ lo then
3:         return (none, −∞)                ▷ fewer than two entries, so no pair exists
4:     end if
5:     m ← ⌊(lo + hi)/2⌋
```

```
    (P_L, v_L) ← Best-Overlap(A, lo, m)
    (P_R, v_R) ← Best-Overlap(A, m + 1, hi)
    (P_C, v_C) ← Best-Crossing(A, lo, m, hi)
    return whichever of (P_L, v_L), (P_R, v_R), (P_C, v_C) has the largest value
end function
```

```
function Best-Crossing(A, lo, m, hi)
    j* ← index in [m + 1 ... hi] with the smallest x value
    best ← −∞
    P ← none
    for i ← lo to m do
        v ← max(0, y_i − max(x_i, x_{j*}) + 1)                    ▷ by (∗)
        if v > best then
            best ← v
            P ← (i, j*)
        end if
    end for
    return (P, best)
end function
```

Why it works. The proof is by induction on the number of entries in the subarray.

Base case. If $hi \le lo$ the subarray holds at most one entry, so it contains no pair, and the maximum over the empty set is $-\infty$, which is what the algorithm returns. This also guarantees the base case never wins a comparison higher up in the recursion.

Inductive step. Assume the two recursive calls correctly return the best pair within $L = A[lo \dots m]$ and within $R = A[m+1 \dots hi]$. Every pair of distinct indices of $A[lo \dots hi]$ has both indices in L, both indices in R, or one in each, and these three cases are exhaustive and mutually exclusive. The first two are covered by the inductive hypothesis and the third by Best-Crossing, so the final comparison returns the maximum over all pairs of $A[lo \dots hi]$.

Best-Crossing is correct by the two facts above. Write a crossing pair as (i, j) with $i \in L$ and $j \in R$; then $i < j$ automatically, since every index of L is smaller than every index of R, so $(*)$ applies to it. By the second fact, $\text{overlap}(i, j^*) = \max_{j \in R} \text{overlap}(i, j)$ for each $i \in L$, so the loop computes

$$\max_{i \in L} \text{overlap}(i, j^*) \;=\; \max_{i \in L} \; \max_{j \in R} \; \text{overlap}(i, j),$$

which is the best crossing pair.

The smallest subarray containing a pair has two entries. There both recursive calls return $-\infty$ and the single pair is found by Best-Crossing, so no pair is lost at the bottom of the recursion.

Runtime. Best-Crossing scans the right half once to find j^*, which is $O(n)$, and then scans the left half once doing $O(1)$ work per entry by part (a), which is also $O(n)$. So the combine step is $O(n)$, and

$$T(n) = 2T\Big(\frac{n}{2}\Big) + O(n).$$

By the Master Theorem with $a = 2$, $b = 2$, $d = 1$, we have $a = b^d$, so

$$T(n) = O(n^d \log n) = O(n \log n).$$

Answer: Recurse on both halves, and for the crossing case use the single index $j^* \in R$ with the smallest x value, computing overlap(i, j^*) for every $i \in L$. This gives $T(n) = 2T(n/2)+O(n) = \boxed{O(n \log n)}$.

6. Suppose you are given a collection of n designer watches, which all look the same. More than half of the watches are genuine. All genuine watches are made of the same materials, while counterfeit watches may be made of the same or different materials.

You have a machine that compares two watches and determines whether they are made of the same materials, but cannot directly determine whether either watch is genuine.

(a) Suppose we examine watches one by one. For each watch, we compare it with the other $n-1$ watches to determine whether it is genuine. Once we find a genuine watch, we can identify all genuine watches.

What is the worst-case number of comparisons in Θ notation?

Solution.

In the worst case, we examine all counterfeit watches before finding a genuine watch.

Since more than half the watches are genuine, there can be at most

$$\left\lfloor \frac{n-1}{2} \right\rfloor$$

counterfeit watches.

Each examined watch requires $n-1$ comparisons.

Thus,

$$T(n) = \left(\left\lfloor \frac{n-1}{2} \right\rfloor + 1 \right) (n-1) = \Theta(n^2).$$

Answer: $\boxed{\Theta(n^2)}$.

(b) Suppose we avoid repeating comparisons by remembering the results of previous comparisons. Does this improve the worst-case Θ runtime?

Solution.

No. Although previously performed comparisons can be reused, we may still need to examine approximately $n/2$ watches.

The first examined watch requires $n-1$ comparisons, the second requires at most $n-2$ new comparisons, and so on.

Let $m = \lfloor (n-1)/2 \rfloor$ be the maximum number of counterfeit watches.

In the worst case, we examine these m watches first, performing

$$T(n) \geq \sum_{k=1}^{m} (n-k).$$

Since $m = \Theta(n)$,

$$\sum_{k=1}^{m} (n-k) = \Theta(n^2).$$

For example, ignoring lower-order terms,

$$T(n) \approx \sum_{k=1}^{n/2} (n-k) = \frac{3n^2}{8} - \frac{n}{4} = \Theta(n^2).$$

Since there are at most $\binom{n}{2} = O(n^2)$ distinct comparisons, the worst-case runtime is $\Theta(n^2)$.

Answer: No. Avoiding repeated comparisons improves the constant factor, but the worst-case runtime remains $\boxed{\Theta(n^2)}$.

(c) Consider a randomized algorithm that repeatedly selects a watch uniformly at random and compares it with all other $n-1$ watches. If the selected watch is genuine, the algorithm terminates; otherwise, it repeats.

What is the worst-case expected number of comparisons? Give an exact answer as a function of n.

Solution.

Let g be the number of genuine watches.

The probability of selecting a genuine watch is

$$p = \frac{g}{n}.$$

The number of trials until success follows a geometric distribution, so

$$E[\text{trials}] = \frac{1}{p} = \frac{n}{g}.$$

Each trial performs exactly $n-1$ comparisons.

Therefore, the expected number of comparisons is

$$E[C] = (n-1)\frac{n}{g} = \frac{n(n-1)}{g}.$$

Approximate worst-case answer:

Since more than half the watches are genuine,

$$p > \frac{1}{2}.$$

Using $p \approx 1/2$ gives

$$E[C] \approx (n-1)\frac{1}{1/2} = \boxed{2(n-1)}.$$

More precisely,

$$E[C] < 2(n-1),$$

so the expected runtime is $\Theta(n)$ in the worst case over valid inputs.

Exact worst-case answer:

The expected number of comparisons is maximized when the number of genuine watches is minimized.

Because strictly more than half are genuine,

$$g_{\min} = \left\lfloor \frac{n}{2} \right\rfloor + 1.$$

Substituting,

$$\boxed{E[C]_{\max} = \frac{n(n-1)}{\lfloor n/2 \rfloor + 1}}.$$

(d) Design an algorithm that identifies all genuine watches using $o(n^2)$ comparisons.

Solution.

Throughout the pseudocodes, $\textsc{Same}(x, y)$ returns true if watches x and y are made of the same materials. Additionally, the array of watches is denoted as W and the size of W is denoted as $|W|$.

Approach 1: Divide-and-Conquer

If a material is the majority of a segment, it must also be the majority of at least one of the segment's two halves.

Otherwise, if the material appeared in at most half of each half, it could not appear in more than half of the entire segment.

Therefore, we can recursively find the majority candidate of each half and check whether either candidate is the majority of the entire segment.

```
1: function SOLVE(W)
2:     if |W| = 1 then
3:         return W[1]                          ▷ A single watch is its own majority
4:     end if
5:     Split W into two approximately equal halves L, R      ▷ Divide into two subproblems
6:     a ← SOLVE(L)
7:     b ← SOLVE(R)                             ▷ Find each half's majority candidate
                                                ▷ The overall majority must be among a, b
8:     for each distinct non-None candidate c ∈ {a, b} do
9:         count ← 0
10:        for each watch x ∈ W do
11:            if SAME(c, x) then
12:                count ← count + 1
13:            end if
14:        end for
                                                ▷ Verify majority in the entire segment
15:        if count > |W|/2 then
16:            return c
17:        end if
18:    end for
19:    return None                              ▷ No majority exists in this segment
20: end function
```

After calling solve(W) on the entire collection, we obtain a genuine watch, since more than half the watches are genuine.

We then compare this watch with every other watch to identify all genuine watches.

Runtime:

We recursively solve two subproblems of size approximately $n/2$. Checking the two candidates requires at most $O(n)$ comparisons.

Thus,

$$T(n) = 2T(n/2) + O(n).$$

By the Master Theorem,

$$T(n) = \Theta(n \log n).$$

The final pass to identify all genuine watches requires another $n - 1$ comparisons, which does not change the asymptotic runtime.

Therefore,

$$\boxed{T(n) = \Theta(n \log n) = o(n^2)}.$$

Approach 2: Boyer–Moore Majority Voting

Maintain one candidate watch and a counter. A matching material increases the counter, while a different material decreases it. A decrease can be viewed as canceling a pair of watches made from different materials. When the counter reaches zero, choose the next watch as a new candidate. Because genuine watches form a strict majority, their material survives this cancellation process. Finally, compare the surviving candidate with every watch to identify the entire genuine group.

```
1:  function SOLVE(W)
2:      c ← None                                   ▷ Candidate genuine watch
3:      count ← 0
4:      for all w ∈ W do
5:          if count = 0 then
6:              c ← w; count ← 1                   ▷ Start a new candidate group
7:          else if SAME(c, w) then
8:              count ← count + 1                  ▷ Matching material reinforces candidate
9:          else
10:             count ← count − 1                  ▷ Different materials cancel
11:         end if
12:     end for
13:     G ← {c}                                    ▷ Collect all watches matching the majority
14:     for all w ∈ W with w ≠ c do
15:         if SAME(c, w) then
16:             G ← G ∪ {w}
17:         end if
18:     end for
19:     return G
20: end function
```

Runtime. The first pass uses at most $n-1$ material comparisons, and the final pass uses $n-1$ comparisons. Hence

$$T(n) \le 2(n-1), \qquad \boxed{T(n) = \Theta(n) = o(n^2)}.$$

Algorithm	**Worst-Case Runtime**
Divide-and-Conquer	$\Theta(n \log n)$
Boyer–Moore Majority Vote	$\Theta(n)$

7. You flip a fair coin repeatedly and stop as soon as either you've gotten a total of two heads (not necessarily in a row) or you've made six flips, whichever happens first. Let X be the total number of flips made.

(a) For each possible value of X (from 2 to 6), find $\Pr(X = x)$ by summing the probabilities of every outcome sequence giving that value of X. Use this to compute $E[X]$.

Solution. Each sequence of k flips has probability $1/2^k$. For $x \le 5$, we stop at flip x when it is the second head, so flip x is H and exactly one earlier flip is H. For $x = 6$, the first 5 flips have at most one head (so we never stopped early), and we always stop after the 6th flip whether it is H or T.

x	Outcome sequences	$\Pr(X = x)$
2	HH	$1/4$
3	HTH, THH	$2/8 = 1/4$
4	HTTH, THTH, TTHH	$3/16$
5	HTTTH, THTTH, TTHTH, TTTHH	$4/32 = 1/8$
6	First 5 flips: TTTTT, HTTTT, THTTT, TTHTT, TTTHT, TTTTH 6th flip: H or T ($6 \times 2 = 12$ sequences, each with probability $1/64$)	$12/64 = 3/16$

Therefore

$$E[X] = 2 \cdot \frac{1}{4} + 3 \cdot \frac{1}{4} + 4 \cdot \frac{3}{16} + 5 \cdot \frac{1}{8} + 6 \cdot \frac{3}{16} = \frac{8 + 12 + 12 + 10 + 18}{16} = \boxed{\frac{15}{4}}.$$

(b) For $j = 1, \ldots, 6$, let $I_j = 1$ if a j-th flip occurs (i.e., we have fewer than 2 heads among the first $j-1$ flips), and 0 otherwise. Express X in terms of the I_j's, compute $E[I_j]$ for each j, and use linearity of expectation to recompute $E[X]$. Confirm it matches part (a).

Solution. Each flip that occurs adds 1 to X, so $X = I_1 + I_2 + \cdots + I_6$.

$E[I_j] = \Pr(\text{at most one head in the first } j-1 \text{ flips})$. Out of the 2^{j-1} sequences, 1 has no heads and $j-1$ have exactly one head, so $E[I_j] = j/2^{j-1}$:

j	1	2	3	4	5	6
$E[I_j]$	1	1	3/4	1/2	5/16	3/16

By linearity of expectation,

$$E[X] = \sum_{j=1}^{6} E[I_j] = \frac{16+16+12+8+5+3}{16} = \boxed{\frac{15}{4}},$$

which matches part (a).